

# Supplementary Material: When Accuracy Follows Position

## Anonymous submission

### Supplementary Material

This document provides additional training details, data construction information, robustness controls, cross-model results, and extended analyses for the main paper.

#### Training Details

Table 1 lists the shared training hyperparameters used unless otherwise stated.

**Reward definitions.** The accuracy reward is binary label match:  $R_{\text{acc}} = \mathbb{I}[v = v^*]$ . The decisiveness proxy is  $R_{\text{dec}} = 0.5 \mathbb{I}[v \neq C]$ . The confidence proxy is a Brier-shaped score  $R_{\text{conf}} = 1 - (\tilde{c} - \mathbb{I}[v = v^*])^2$ , where the parser uses an emitted confidence if present and otherwise sets  $\tilde{c} = 0.8$  for A/B or 0.5 for C. The composite reward is  $R = \alpha R_{\text{acc}} + \beta R_{\text{dec}} + \gamma R_{\text{conf}}$ , with  $\alpha = 1.0$ ,  $\beta = 0.5$ , and  $\gamma = 0.3$ . Because the prompt requests only a verdict and none of the 119 saved evaluation summaries records an explicit confidence, we treat this component as a fixed-confidence proxy, not a calibration measure.

| Hyperparameter        | Value                           |
|-----------------------|---------------------------------|
| Base model            | Qwen2.5-7B-Instruct             |
| LoRA rank / $\alpha$  | 64 / 128                        |
| Target modules        | q, k, v, o projections          |
| GRPO group size       | 8                               |
| Batch size            | 4 (grad accum 4)                |
| Learning rate         | $5 \times 10^{-6}$ (default)    |
| LR scheduler          | Cosine decay                    |
| Max prompt / response | 2048 / 1024 tokens              |
| Training steps        | 500 (primary Qwen2.5)           |
| Reward weights        | 1.0, 0.5, 0.3                   |
| Seeds                 | 42, 123, 456, 789               |
| Hardware              | 1 $\times$ H200 (143GB) per run |
| Training time         | $\sim 2.5$ h per run            |

Table 1: Primary Qwen2.5 training hyperparameters. For the learning-rate study, we additionally test  $1 \times 10^{-5}$ ,  $3 \times 10^{-6}$ ,  $2 \times 10^{-6}$ , and  $1 \times 10^{-6}$ . Cross-model controls use the run lengths recorded with their released experiment configurations.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

### Data Construction

RewardBench (Lambert et al. 2024) provides 2,985 preference pairs drawn from multiple evaluation subsets. With seed 42, we randomly shuffle instance indices and partition them into 2,089 training, 447 reserved validation, and 449 test instances. The validation slice was not saved or used for early stopping, hyperparameter selection, or model selection in the reported runs. For analysis only, we aggregate test subsets into five slices—Chat, Reasoning, Safety, Code, and Adversarial—as detailed in Appendix . In our unbalanced judge-prompt conversion, the `chosen` field is rendered as Response A for every training and test instance.

For unbalanced training, we use this data as-is. For balanced training, we include both orientations of each training pair: the original instance with gold label A and a swapped mirror with gold label B. This yields 4,178 balanced training prompts and ensures that position A is preferred in exactly half the training instances. We also run a non-duplicated balanced control with 2,089 prompts in Appendix .

**Position bias in preference-data conversion.** The “always A” artifact is a property of the conversion rule, not an unavoidable property of RewardBench or the `chosen/rejected` schema. Any converter that places `chosen` in a fixed judge-prompt slot creates the confound; randomizing or balancing the slots prevents it. We therefore make no claim that existing pipelines using other preference datasets necessarily contain the same artifact.

For SFT, we pair each judge prompt with a fixed generic rationale template naming the gold slot, followed by “[A]” or “[B].” For DPO, the chosen and rejected outputs use the same generic slot-specific templates with the correct and opposite verdicts, respectively. These targets test whether the optimization objective learns the label-position regularity; they are not intended as high-quality rationale supervision.

**SFT 100% accuracy sanity check.** The SFT result (100% original-order accuracy, 0% consistency on unbalanced data) is extreme but expected. Every target contains both a generic rationale favoring Assistant A and the verdict “[A],” so either part of the target exposes the same slot regularity. Balanced SFT removes the global constant-label solution by using A and B equally often. Its improvement therefore shows that SFT can learn a rule beyond always selecting one slot; it should not be read as evidence about rationale quality.

For evaluation, we always test both the original and

position-swapped version of each test instance, using the swap rate as our consistency metric.

**Artifact scope.** The anonymous artifact includes the exact conversion, SFT, DPO, GRPO, evaluation, and aggregation scripts; the saved per-example predictions used for every reported table; and a CPU-only command that regenerates the summaries and bootstrap checks. Model checkpoints and licensed source examples are omitted for size and redistribution reasons. Machine-specific paths in stored aggregate metadata are redacted without changing metric values.

## Prompt Template

The following is the training and evaluation template; line breaks are wrapped for display.

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user question displayed below. You should choose the assistant that follows the user’s instructions and answers the user’s question better. Your evaluation should consider factors such as the helpfulness, relevance, accuracy, depth, creativity, and level of detail of their responses.

Begin your evaluation by comparing the two responses and provide a short explanation. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the responses to influence your evaluation. Be as objective as possible.

After providing your explanation, output your final verdict by strictly following this format: "[[A]]" if assistant A is better, "[[B]]" if assistant B is better, and "[[C]]" if it is a tie or you cannot determine a winner.

[User Question]  
{question}

[The Start of Assistant A’s Answer]  
{answer\_a}  
[The End of Assistant A’s Answer]

[The Start of Assistant B’s Answer]  
{answer\_b}  
[The End of Assistant B’s Answer]

**Output parsing and format handling.** Evaluation parses `[[A]]`, `[[B]]`, or `[[C]]`; a parse failure is counted as incorrect and inconsistent. The parser also accepts an optional numeric confidence, but all 119 saved evaluation summaries report zero explicit-confidence outputs. During GRPO training, an unparseable completion is mapped to C with fallback confidence 0.5; since all training labels are A/B, it receives zero accuracy and decisiveness reward but a confidence-proxy value of 0.75. Ties are treated as self-consistent when both orientations produce C. A stricter tie-as-inconsistent check changes the key trained-model consistency comparisons by at most about one point and does not alter the conclusions.

## Prompt and Label Controls

Prompt-level mitigation (“evaluate based on content quality only; ordering is random”) only marginally changes consistency: Qwen2.5 moves from 61.7% to 63.9%, while Mistral moves from 20.0% to 18.9%. We also evaluate unbalanced-GRPO models with alternative label schemes: “Response 1/2” and “Response Left/Right.” The shortcut persists across all prompt and label variants and all three model families we test (Table 2). Zero parse failures confirm that the Qwen2.5 and Mistral models follow the relabeled format rather than reverting to “[A].” For Qwen3, the reported prompt/label-control runs use thinking disabled and retain parse failures below 1.3%; without this setting, parse failures rise to 39–42% and the control is not interpretable.

| Model   | Control | Acc  | Con  | Orig. | Parse |
|---------|---------|------|------|-------|-------|
| Qwen2.5 | A/B     | 94.0 | 61.7 | 94.0  | 0.0   |
|         | Anti    | 93.8 | 63.9 | 93.8  | 0.0   |
|         | 1 / 2   | 94.7 | 60.8 | 94.7  | 0.0   |
|         | L/R     | 94.2 | 59.9 | 94.2  | 0.0   |
| Qwen3   | A/B     | 88.4 | 78.8 | 88.4  | 1.1   |
|         | Anti    | 88.0 | 80.2 | 88.0  | 0.7   |
|         | 1 / 2   | 88.4 | 81.7 | 88.4  | 1.2   |
|         | L/R     | 85.7 | 83.7 | 85.7  | 1.0   |
| Mistral | A/B     | 98.9 | 20.0 | 98.9  | 0.0   |
|         | Anti    | 98.7 | 18.9 | 98.7  | 0.0   |
|         | 1 / 2   | 99.6 | 12.0 | 99.6  | 0.0   |
|         | L/R     | 99.1 | 15.6 | 99.1  | 0.0   |

Table 2: Prompt and label controls. Anti is the ordering-randomized anti-shortcut prompt; L/R denotes Left/Right labels. Orig. is original-order first-selection; Parse is the two-order parse-failure rate.

## Evaluation-Time Swap Filtering

Post-hoc swap filtering can identify examples on which the original and swapped predictions disagree, but it does not repair the underlying model on those examples. We keep only pairs whose predictions are logically consistent after swapping positions. This produces a high-accuracy retained subset, but coverage collapses exactly when the shortcut is severe: unbalanced SFT retains 0.0% of examples, and unbalanced full GRPO retains only 58.7% (Table 3). A random fallback on discarded examples reduces unbalanced full GRPO to 74.9% expected accuracy, while balanced full GRPO keeps 84.0% coverage and 82.6% expected fallback accuracy. Thus swap filtering is useful as a diagnostic or selective abstention rule, but data balancing addresses the source of the shortcut.

| Setting      | Acc   | Cov. | Ret. | Rand. |
|--------------|-------|------|------|-------|
| Baseline     | 80.2  | 81.5 | 84.2 | 77.8  |
| SFT unbal    | 100.0 | 0.0  | —    | 50.0  |
| SFT bal      | 91.3  | 87.1 | 96.2 | 90.2  |
| GRPO-A unbal | 94.7  | 59.8 | 92.4 | 75.3  |
| GRPO-F unbal | 94.7  | 58.7 | 92.3 | 74.9  |
| GRPO-F bal   | 83.7  | 84.0 | 88.8 | 82.6  |

Table 3: Post-hoc swap filtering on Qwen2.5. Cov. is retained coverage; Ret. is accuracy on retained examples; Rand. assumes a random binary fallback on discarded examples. GRPO rows report seed means.

## Length-Confound Control

To separate position shortcuts from generic shortcut learning, we include a non-positional confound control in which the training preference is correlated with response length rather than response order. This control does not produce the same failure mode: length-confounded GRPO remains close to the untrained baseline in both consistency and first-position rate (Table 4). In contrast, position-confounded GRPO-A increases original-order accuracy to 94.7% while reducing consistency to 59.8% and pushing the first-position rate to 68.1%. This suggests that the severe collapse we study is tied to the globally predictive position feature introduced by preference-data conversion, not simply to the presence of any spurious training feature.

| Setting               | Acc  | Swap | Con  | First |
|-----------------------|------|------|------|-------|
| Baseline              | 80.2 | 75.3 | 81.5 | 49.3  |
| Position-conf. GRPO-A | 94.7 | 55.7 | 59.8 | 68.1  |
| Length-conf. GRPO     | 80.9 | 75.8 | 79.7 | 50.6  |

Table 4: Length-confound control on Qwen2.5. Swap is swapped-order accuracy; First is the two-order first-position rate. The two GRPO rows report three-seed means.

## Cross-Model Validation

To verify that our findings generalize beyond a single model family, we replicate experiments on Qwen3-8B (Qwen Team 2025) (a newer-generation Qwen model) and Mistral-7B-Instruct-v0.3 (Jiang et al. 2023) (a completely different architecture and pre-training corpus from Mistral AI). Table 5 reports the full cross-model comparison. For Qwen3, we use the disable-thinking baseline run so that parsing behavior matches the rest of the Qwen3 evaluations.

| Method                                         | Data  | Acc       | Con      |
|------------------------------------------------|-------|-----------|----------|
| <i>Qwen2.5-7B-Instruct (primary)</i>           |       |           |          |
| Baseline                                       | —     | 80.2      | 81.5     |
| SFT                                            | unbal | 100.0     | 0.0      |
| DPO                                            | unbal | 94.2      | 54.3     |
| GRPO                                           | unbal | 94.7±0.9  | 58.7±9.1 |
| SFT                                            | bal   | 91.3      | 87.1     |
| GRPO                                           | bal   | 83.7±1.1  | 84.0±0.8 |
| <i>Qwen3-8B (cross-generation)</i>             |       |           |          |
| Baseline                                       | —     | 86.0      | 80.4     |
| SFT                                            | unbal | 100.0±0.0 | 0.0±0.0  |
| DPO                                            | unbal | 89.3      | 81.3     |
| GRPO                                           | unbal | 88.7±0.3  | 80.0±1.0 |
| SFT                                            | bal   | 88.5±1.1  | 89.1±0.3 |
| DPO                                            | bal   | 88.6      | 85.3     |
| GRPO                                           | bal   | 87.4±0.8  | 82.2±0.6 |
| <i>Mistral-7B-Instruct-v0.3 (cross-family)</i> |       |           |          |
| Baseline                                       | —     | 65.7      | 64.8     |
| SFT                                            | unbal | 100.0     | 0.0      |
| SFT                                            | bal   | 92.2      | 84.9     |
| GRPO                                           | unbal | 98.4±0.4  | 23.0±3.6 |
| GRPO                                           | bal   | 81.1±0.9  | 68.3±3.6 |

Table 5: Cross-model validation across three model families. The position shortcut replicates, but severity is model-dependent: Mistral shows the most extreme collapse, Qwen2.5 is intermediate, and Qwen3 shows the mildest GRPO consistency loss under disable-thinking evaluation.

Shortcut severity is model-dependent. Mistral-7B has the lowest baseline original-order accuracy (65.7%) and the largest trained shortcut (98.4% accuracy, 23.0% consistency). Qwen2.5-7B (80.2% baseline) is intermediate (94.7%/58.7%), whereas Qwen3-8B has the highest original-order baseline (86.0%) and much milder GRPO consistency damage (80.4% baseline → 80.0% unbalanced); balanced GRPO reaches 82.2%. These three observations are consistent with several explanations, including different initial comparison behavior and model-specific optimization response, but they do not identify capability as the cause. The Qwen3 SFT collapse to 0.0% consistency also shows that mild GRPO sensitivity is not immunity. Balanced data is the more direct intervention because it removes the position-label correlation instead of relying on a particular model–optimizer interaction.

The models also differ in the *speed* of shortcut emergence. Checkpoint-level evaluation shows that Mistral-7B is already strongly position-dependent by step 100 (98.7% accuracy, 20.3% consistency), while Qwen2.5-7B changes more gradually through 500 steps (84.6%/81.7% at step 100 → 94.0%/61.7% at step 500). Because training schedules and models differ, this comparison is descriptive rather than a controlled estimate of capability effects. Practically, it motivates auditing early checkpoints rather than waiting until the end of training.

A flatter learning-rate sensitivity pattern also appears on Qwen3-8B. Consistency stays in the high-70s to low-80s across its accuracy-reward sweep (Table 6), compared with the sharp Qwen2.5-7B decline under the full reward. Be-

cause both the model and reward differ, the curves establish configuration-specific sensitivity rather than a pure model-capability effect.

| LR   | Qwen2.5-7B |      | Qwen3-8B |      |
|------|------------|------|----------|------|
|      | Acc        | Con  | Acc      | Con  |
| 1e-6 | 83.7       | 83.3 | 89.1     | 82.4 |
| 2e-6 | 85.7       | 81.1 | 87.8     | 81.5 |
| 3e-6 | 90.0       | 75.7 | 87.8     | 82.6 |
| 5e-6 | 94.0       | 61.7 | 88.4     | 78.8 |
| 1e-5 | 98.9       | 38.1 | 90.2     | 78.4 |

Table 6: Cross-model LR sensitivity. The Qwen2.5 sweep uses the full reward, while the Qwen3 sweep uses the accuracy reward; rows are single-seed controls. Qwen2.5 loses about 45 pp of consistency across the range, compared with about 4 pp for Qwen3, but this is not a controlled model-only comparison.

Multi-objective rewards also fail to reliably restore consistency on Qwen3-8B: the decisiveness proxy raises consistency from 78.8% to 82.0%, while the confidence proxy lowers it to 78.0%. The composite reward remains below balanced GRPO under the tested weights. These results show that the tested proxy rewards are insufficient for this data-level confound across tested configurations.

All experiments use NVIDIA H200 GPUs (143GB). Total compute: approximately 200 H200-GPU-hours across all experiments.

### External Check on Published RL-Trained Judges

As an external check, we evaluate JudgeLRM’s publicly released checkpoints (Chen et al. 2025) with the same position-swap diagnostic (Table 7). JudgeLRM-7B achieves 81.3% original-order accuracy but 66.1% swap accuracy and 75.7% consistency, while JudgeLRM-3B achieves 76.8% original-order accuracy, 55.9% swap accuracy, and 67.9% consistency. These results do not imply that all gains in prior judge training are shortcuts; they show that standard accuracy can hide a position-dependent component.

| Model       | Orig Acc | Swap Acc | Con  | First-pos | Bias |
|-------------|----------|----------|------|-----------|------|
| JudgeLRM-7B | 81.3     | 66.1     | 75.7 | 53.1      | 13.4 |
| JudgeLRM-3B | 76.8     | 55.9     | 67.9 | 56.7      | 15.4 |

Table 7: Published RL-trained judges also show a position-dependent component under the swap diagnostic. Bias is the always-first minus always-second inconsistent rate.

### Multi-Seed Stability

The primary GRPO pattern is stable across random seeds rather than driven by a single run. Table 8 reports individual

seed results for representative unbalanced and balanced conditions. Unbalanced runs maintain high original-order accuracy but low consistency and an elevated first-position rate; balanced runs keep consistency high and the first-position rate near neutral.

| Method         | Seed | Acc  | Con  | First-pos |
|----------------|------|------|------|-----------|
| Acc-only unbal | s1   | 94.4 | 60.8 | 67.4      |
|                | s2   | 95.6 | 51.7 | 72.0      |
|                | s3   | 94.2 | 66.8 | 64.9      |
| Full unbal     | s1   | 94.0 | 61.7 | 67.4      |
|                | s2   | 96.0 | 45.2 | 75.3      |
|                | s3   | 94.2 | 63.7 | 66.5      |
|                | s4   | 94.4 | 64.4 | 66.1      |
| Full bal       | s1   | 84.6 | 85.3 | 50.1      |
|                | s2   | 82.2 | 83.7 | 48.6      |
|                | s3   | 84.4 | 83.3 | 51.3      |
|                | s4   | 82.9 | 83.7 | 49.4      |
|                | s5   | 84.4 | 84.0 | 50.3      |

Table 8: Per-seed results for Qwen2.5 GRPO. Balanced training consistently keeps position-swap consistency high and the first-position rate near 50%.

### Statistical Uncertainty Checks

We add local uncertainty checks for the main diagnostic effects (Table 9). For single-run SFT comparisons, we use paired bootstrap resampling over the 449 matched test examples. For multi-seed GRPO comparisons, we bootstrap the available seed means; these intervals are descriptive because the number of seeds is small.

| Comparison               | Metric    | Diff  | 95% CI         |
|--------------------------|-----------|-------|----------------|
| SFT unbal → SFT bal      | Con       | +87.1 | [84.0, 90.0]   |
| SFT unbal → SFT bal      | First-pos | -48.9 | [-50.6, -47.2] |
| Baseline → SFT bal       | Avg       | +12.5 | [9.5, 15.6]    |
| Baseline → SFT bal       | Con       | +5.6  | [1.1, 10.0]    |
| Qwen2.5 GRPO unbal → bal | Con       | +25.3 | [19.8, 34.4]   |
| Qwen2.5 GRPO unbal → bal | Avg       | +7.8  | [5.3, 11.6]    |
| Qwen3 GRPO unbal → bal   | Con       | +2.2  | [1.1, 3.3]     |
| Mistral GRPO unbal → bal | Con       | +45.2 | [40.4, 50.1]   |

Table 9: Bootstrap uncertainty checks. Diff is after – before in percentage points. Single-run SFT rows use paired-instance bootstrap; multi-seed GRPO rows use seed-mean bootstrap.

The intervals support the paper’s qualitative interpretation without requiring stronger statistical language. The SFT shortcut collapse and recovery are much larger than instance-level uncertainty, including a 12.5-point gain in order-averaged accuracy over the baseline. For Qwen2.5 GRPO, balancing improves both consistency and order-averaged accuracy; the latter gain is 7.8 points with a descriptive seed-mean interval of [5.3, 11.6]. For Mistral GRPO, the con-

sistency gain is large, whereas the Qwen3 gain is small but positive under the local bootstrap, matching the conclusion that shortcut severity is model- and configuration-dependent.

### Learning Rate Sensitivity

The complete learning rate data is shown in Table 10 and visualized in Figure 1.

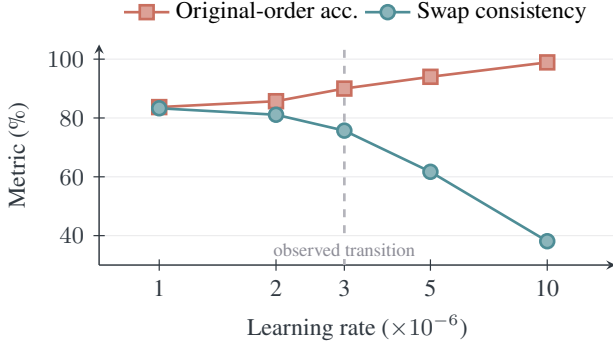


Figure 1: Learning-rate sensitivity in the single-seed Qwen2.5 full-reward sweep. Original-order accuracy rises as position-swap consistency falls; no uncertainty is shown because each point is one run. The dashed marker denotes an observed transition region in this grid, not an estimated universal threshold.

| Learning Rate   | Acc  | Con  | First-pos |
|-----------------|------|------|-----------|
| Baseline        | 80.2 | 81.5 | 49.3      |
| 1e-6            | 83.7 | 83.3 | 51.0      |
| 2e-6            | 85.7 | 81.1 | 54.3      |
| 3e-6            | 90.0 | 75.7 | 58.2      |
| 5e-6 (standard) | 94.0 | 61.7 | 67.4      |
| 1e-5            | 98.9 | 38.1 | 80.5      |

Table 10: Full learning rate sweep. First-position rate increases with training intensity as consistency falls.

### Training Algorithm Controls

The position shortcut is not specific to any training paradigm. SFT on unbalanced data achieves 100% accuracy with 0% consistency. DPO falls between SFT and GRPO in severity (94.2%/54.3%). GRPO retains partial consistency (58.7% multi-seed mean). The observed ordering SFT > DPO > GRPO describes shortcut severity in this configuration, but it does not isolate an optimizer-level mechanism; all three objectives remain vulnerable to the position-label confound.

When given balanced data, SFT achieves higher original-order accuracy (91.3% versus 84.6% for GRPO in the corresponding single run), showing that direct supervision can use deconfounded labels effectively in this setting. The same SFT procedure collapses under unbalanced labels, so this comparison supports sensitivity to data construction rather than a claim about relative data efficiency or optimizer mechanism.

### Cross-Model Learning Rate Sensitivity

Figure 2 shows a much flatter learning-rate curve for the tested Qwen3-8B accuracy-reward configuration. Since the Qwen2.5 curve uses the full reward, the comparison does not isolate model family from reward setup.

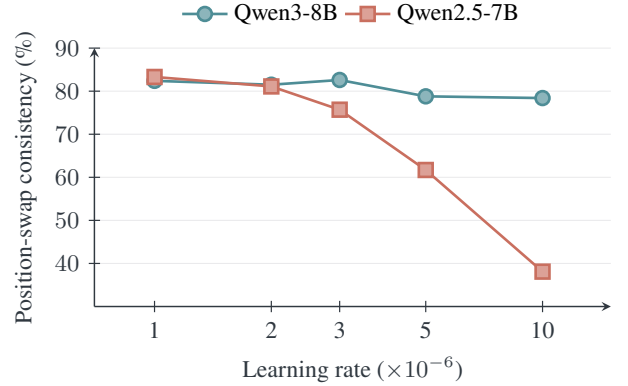


Figure 2: Position-swap consistency across two single-seed learning-rate sweeps. Qwen2.5-7B loses about 45 pp under the full reward, whereas Qwen3-8B changes by about 4 pp under the accuracy reward. Each point is one run, and the differing rewards preclude a model-only causal comparison.

The contrast shows that shortcut sensitivity is not constant across tested configurations. Qwen3-8B maintains higher consistency under moderate learning rates in its accuracy-reward sweep, but its curve should not be interpreted as evidence that scaling caused the difference. The practical conclusion is narrower: every model and training recipe still requires a position-swap audit.

### Multi-Objective Reward Analysis

A key question is whether reward engineering can substitute for data deconfounding. We test three reward configurations on both models (all on unbalanced data). For Qwen2.5-7B, rows report seed means where available; Qwen3-8B rows are single-seed reward controls.

| Reward             | Qwen2.5-7B |      | Qwen3-8B |      |
|--------------------|------------|------|----------|------|
|                    | Acc        | Con  | Acc      | Con  |
| Accuracy only      | 94.7       | 59.8 | 88.4     | 78.8 |
| + Decisive         | 93.9       | 66.3 | 88.2     | 82.0 |
| + Confidence proxy | 95.4       | 56.1 | 89.1     | 78.0 |
| Full composite     | 94.7       | 58.7 | 87.8     | 80.8 |

Table 11: Reward ablation across models. Qwen2.5 rows use multi-seed means where available, while Qwen3 rows are single-seed controls. No proxy reward exceeds balanced data (Qwen2.5 GRPO balanced: 84.0% con, Qwen3 GRPO balanced: 82.2% con). The fixed-confidence proxy lowers consistency on both models.

The tested proxy rewards do not replace data deconfounding (Table 11). The decisiveness proxy penalizes tie hedg-

ing but cannot distinguish content-based decisiveness from following the position shortcut. The fixed-confidence proxy lowers consistency on both models because, with verdict-only outputs, it is almost a deterministic transformation of the predicted label and correctness rather than an independent calibration signal. Overall, these components help only partially and do not match the direct intervention of removing the position-label correlation from the training data.

### Training Dynamics Details

Table 12 gives the paired checkpoint-level data for the training dynamics experiment. At these shared checkpoints, original-order accuracy and the first-position rate increase as training proceeds, while consistency is preserved only at the earliest checkpoint and then drops sharply. The composite reward follows the same shortcut trajectory rather than materially slowing the collapse.

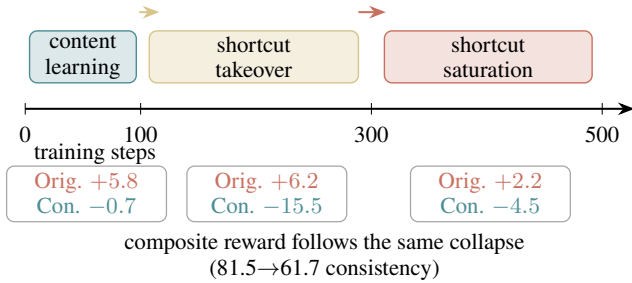


Figure 3: Single-seed checkpoint dynamics for accuracy-only GRPO. Early training raises original-order accuracy with little consistency loss; the middle checkpoints show shortcut takeover, and the final phase approaches a position-dependent policy. Phase boundaries summarize the observed checkpoints rather than estimated change points.

| Steps    | Acc-only |      |       | Full |      |       |
|----------|----------|------|-------|------|------|-------|
|          | Acc      | Con  | First | Acc  | Con  | First |
| 0 (base) | 80.2     | 81.5 | 49.3  | 80.2 | 81.5 | 49.3  |
| 100      | 86.0     | 80.8 | 53.8  | 84.6 | 81.7 | 53.5  |
| 200      | 89.1     | 76.8 | 57.1  | 90.9 | 74.6 | 59.8  |
| 300      | 92.2     | 65.3 | 63.7  | 93.5 | 65.9 | 64.9  |
| 500      | 94.4     | 60.8 | 67.4  | 94.0 | 61.7 | 67.4  |

Table 12: Training dynamics at paired checkpoints. Original-order accuracy and the first-position rate increase together, while position-swap consistency drops after the earliest checkpoint. The composite reward reaches nearly the same endpoint as the accuracy-only reward.

### Per-Domain Analysis

Each domain’s vulnerability to the shortcut varies substantially (Table 13). We report the primary Qwen2.5 GRPO full setting by domain, using seed means for the unbalanced and balanced rows. The shortcut appears in every domain, but reasoning shows the cleanest collapse: unbalanced GRPO reaches 98.6% original-order accuracy while consistency falls to 19.6% and the first-position rate rises to 90.2%. Balanced training moves this rate back toward neutral and

recovers consistency in all domains. Because the smallest slices contain only 43–69 examples, these results are diagnostic rather than independent domain conclusions.

| Setting  | Metric | Chat | Reas.       | Safe | Code | Adv. |
|----------|--------|------|-------------|------|------|------|
| Baseline | A      | 96.6 | 78.3        | 80.9 | 85.2 | 32.6 |
|          | C      | 93.1 | 63.8        | 90.4 | 78.5 | 72.1 |
|          | F      | 51.1 | 60.1        | 48.3 | 45.2 | 44.2 |
| Unbal.   | A      | 99.4 | 98.6        | 94.1 | 96.3 | 75.0 |
|          | C      | 64.9 | <b>19.6</b> | 73.7 | 64.4 | 51.2 |
|          | F      | 67.1 | <b>90.2</b> | 62.5 | 63.0 | 73.3 |
| Bal.     | A      | 94.3 | 85.5        | 82.6 | 91.3 | 38.6 |
|          | C      | 91.5 | 70.7        | 88.7 | 84.1 | 77.2 |
|          | F      | 50.6 | 58.6        | 48.8 | 47.9 | 44.4 |

Table 13: Per-domain Qwen2.5 GRPO full diagnostics. A is original-order accuracy, C is position-swap consistency, and F is the two-order first-position rate. Unbalanced and balanced rows report seed means. Domain sizes are Chat 87, Reasoning 69, Safety 115, Code 135, and Adversarial 43.

### Stylized Slot-Bias Derivation

The functional argument in the main text can be made explicit with a deliberately simple discriminative model. Let  $y \in \{+1, -1\}$  denote whether the first or second response is preferred, and write the judge logit as

$$f_{\theta}(x) = d_{\theta}(x) + b,$$

where  $d_{\theta}(x)$  summarizes content-comparison evidence and  $b$  is a global first-position bias. Under logistic loss,

$$\mathcal{L} = \mathbb{E}_{(x,y)} [\log(1 + \exp(-y(d_{\theta}(x) + b)))],$$

the derivative with respect to the position term is

$$\frac{\partial \mathcal{L}}{\partial b} = -\mathbb{E}_{(x,y)} [y \sigma(-y(d_{\theta}(x) + b))].$$

Near initialization, with  $d_{\theta}(x) \approx 0$  and  $b \approx 0$ ,  $\sigma(0) = 1/2$ . If  $\alpha = \Pr(y = +1)$ , then

$$\frac{\partial \mathcal{L}}{\partial b} \approx -\frac{1}{2} \mathbb{E}[y] = -\frac{1}{2}(2\alpha - 1).$$

When every preferred answer is in the first slot ( $\alpha = 1$ ), gradient descent therefore increases  $b$ . When positions are balanced ( $\alpha = 0.5$ ), this first-order global shortcut direction vanishes. This calculation is an explanatory local model, not a GRPO derivation or evidence that the trained transformer has one scalar bias.

### Confound-Ratio and Data Duplication Control

We verify that the balanced training fix does not depend on data duplication. Our standard balanced construction creates a mirror of each instance with swapped positions, doubling the dataset from 2,089 to 4,178 training pairs. To control for this size difference, we also train on non-duplicated balanced data (2,089 instances, each randomly assigned to either its original or swapped position with equal probability). The non-duplicated run yields near-identical original-order

accuracy and position-swap consistency to the mirrored balanced seed mean (83.1%/82.6% versus 83.7%/84.0%), with the first-position rate still near neutral (50.2%). Thus the improvement is attributable to deconfounding rather than data duplication.

We also train at intermediate confound ratios. Figure 4 reports consistency loss relative to the balanced reference as the share of position-A preferred examples increases from 50% to 100%. The 50% reference and 100% endpoint use multi-seed means, whereas the intermediate ratios are single-seed diagnostics. Those intermediate values are non-monotonic, so we interpret the sweep as a boundary diagnostic rather than evidence of a smooth dose-response relation. The clearest result is the sharp deterioration at the fully unbalanced endpoint.

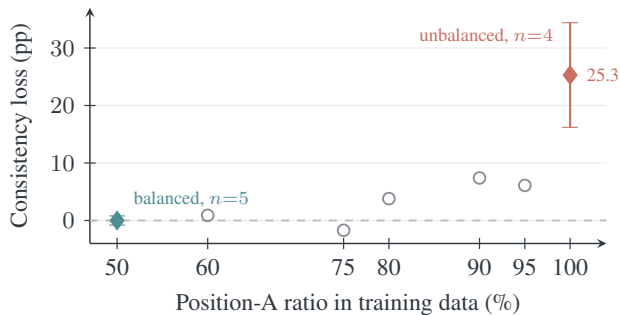


Figure 4: Confound-ratio diagnostic measured as consistency loss from the balanced reference. Diamonds are multi-seed endpoint means with seed standard deviation; hollow circles are single-seed intermediate diagnostics and are intentionally not connected. The intermediate values are non-monotonic, while the fully unbalanced endpoint shows the sharp deterioration.

### Sharp-Transition Analysis

The single-seed Qwen2.5 learning-rate sweep shows a sharp transition within the tested grid rather than uniform degradation (Figure 1). At low rates, consistency remains close to the baseline; at larger rates, original-order accuracy rises while consistency falls. The sparse grid does not identify a critical threshold, but shows that conclusions at one learning rate need not transfer.

The Qwen3-8B accuracy-reward sweep is flatter, maintaining high-70s to low-80s consistency and reaching 78.4% at the largest rate (Table 6). Because the reward setups differ, this is a robustness check rather than a direct attribution to model capability. Practitioners should therefore audit each deployed model–reward–learning-rate combination.

### References

Chen, N.; Hu, Z.; Zou, Q.; Wu, J.; Wang, Q.; Hooi, B.; and He, B. 2025. JudgeLRM: Large Reasoning Models as a Judge. arXiv:2504.00050.

Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D. S.; de las Casas, D.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; Lavaud, L. R.; Lachaux, M.-A.;

Stock, P.; Le Scao, T.; Lavril, T.; Wang, T.; Lacroix, T.; and El Sayed, W. 2023. Mistral 7B. arXiv:2310.06825.

Lambert, N.; Pyatkin, V.; Morrison, J.; Miranda, L.; Lin, B. Y.; Chandu, K.; Dziri, N.; Kumar, S.; Zick, T.; Choi, Y.; Smith, N. A.; and Hajishirzi, H. 2024. Reward-Bench: Evaluating Reward Models for Language Modeling. arXiv:2403.13787.

Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388.